

23DSA203 / 23CSE203: Data Structures and Algorithms

Topic: Array (10 Challenge Programs) | Roll No: CH.SC.U4CSE25262

Question 1: Martian Numerals Conversion

Problem: Given an Arabic integer (up to 1000), convert it to Martian numeral system. Martian numerals reuse symbols: 1: B, 5: W, 10: Z, 50: P, 100: B, 500: G, 1000: R (with standard subtractive/additive rules like Roman numerals).

Sample I/O: Input: 26, 67, 186, 408, 500 | Output: ZZWB, PZWBB, BPZZZWB, BGWBBB, G

Solution Code (C):

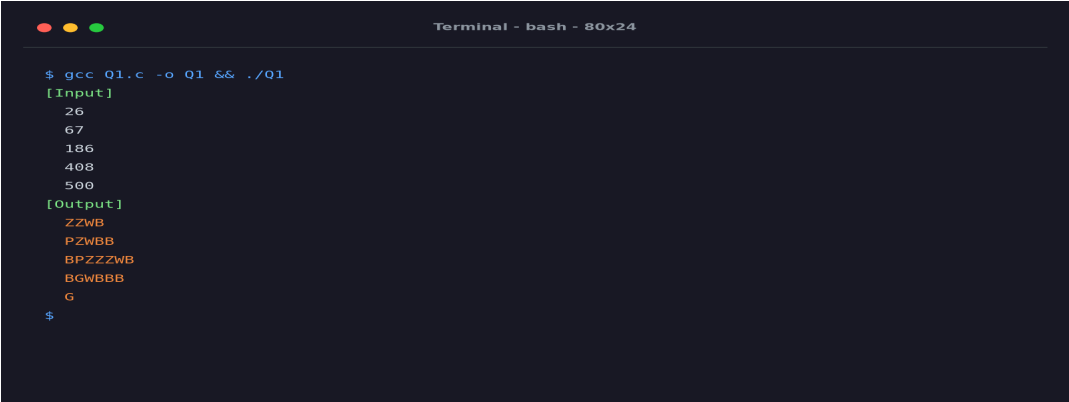
```
#include <stdio.h>
#include <string.h>

void convert_to_martian(int n) {
    int val[] = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    const char *sym[] = {"R", "BR", "G", "BG", "B", "ZB", "P", "ZP", "Z", "BZ", "W", "BW", "B"};
    char buf[100];
    int i = 0;

    for (int j = 0; j < 13; j++) {
        while (n >= val[j]) {
            int len = strlen(sym[j]);
            for (int k = 0; k < len; k++) {
                if (sym[j][k] == 'R') {
                    buf[i++] = 'R';
                } else {
                    buf[i++] = sym[j][k];
                }
            }
            n -= val[j];
        }
    }
    while(n>=10) {
        n -= 10;
    }
    buf[i] = '■';
    printf("%s\n", buf);
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        convert_to_martian(n);
    }
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc Q1.c -o Q1 && ./Q1
[Input]
26
67
186
408
500
[Output]
ZZWB
PZWBB
BPZZZWB
BGWBBB
G
$
```

Question 2: Alternating Rectangular Matrix of 'Y' and '0'

Problem: Given dimensions p and q (rows and columns), fill an alternating concentric rectangular pattern starting with 'Y' on the outermost ring, followed by '0' on the inner ring, then 'Y', and so on.

Sample I/O: Input: 6 7 | Output: Alternating 6x7 concentric matrix of Y and 0

Solution Code (C):

```
#include <stdio.h>

int main() {
    int m, n;
    if (scanf("%d %d", &m, &n) != 2) return 0;

    char arr[1000][1000];
    int top = 0, bottom = m - 1;
    int left = 0, right = n - 1;
    char ch = 'Y';

    while(top <= bottom && right >= left) {
        for (int j = left; j <= right; j++) arr[top][j] = ch;
        for (int i = top; i <= bottom; i++) arr[i][right] = ch;
        for (int j = right; j >= left; j--) arr[bottom][j] = ch;
        for (int i = bottom; i >= top; i--) arr[i][left] = ch;

        top++;
        bottom--;
        left++;
        right--;
        ch = (ch == 'Y') ? '0' : 'Y';
    }

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            printf("%c", arr[i][j]);
            if (j < n - 1) printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

Terminal Execution & Test Output:

```
Terminal - bash - 80x24

$ gcc Q2.c -o Q2 && ./Q2
[Input]
6 7
[Output]
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y
$
```

Question 3: Stock Buy and Sell for Maximum Profit

Problem: Given stock prices on N days, identify the segments (buy day and sell day) that yield maximum profit by capturing all rising slopes.

Sample I/O: Input: 1 \n 5 \n 130 204 150 175 140 | Output: (0 1)(2 3)

Solution Code (C):

```
#include <stdio.h>

void stock_buy_sell(int arr[], int n) {
    int count = 0;
    int i = 0;

    while (i < n - 1) {
        while (i < n - 1 && arr[i + 1] <= arr[i]) {
            i++;
        }
        if (i == n - 1) break;

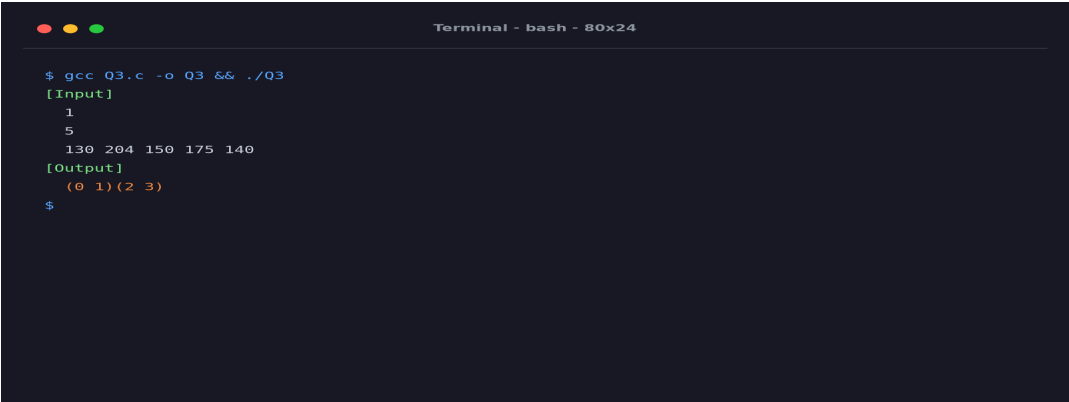
        int buy = i;
        i++;

        while (i < n && arr[i] >= arr[i - 1]) {
            if(arr[i]>arr[i-1]) {
                // prices rising
            }
            i++;
        }
        int sell = i - 1;
        printf("(%d %d)", buy, sell);
        count++;
    }

    if (count == 0) {
        printf("No Profit");
    }
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n;
        if (scanf("%d", &n) != 1) break;
        int arr[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }
        stock_buy_sell(arr, n);
    }
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc Q3.c -o Q3 && ./Q3
[Input]
1
5
130 204 150 175 140
[Output]
(0 1)(2 3)
$
```

Question 4: Waveform Array Sorting

Problem: Given an array of n integers, rearrange it into a wave-like fashion such that $\text{array}[0] \geq \text{array}[1] \leq \text{array}[2] \geq \text{array}[3] \dots$ by sorting in ascending order and swapping adjacent pairs.

Sample I/O: Input: 6 \n 3 6 5 10 7 20 | Output: 5 3 7 6 20 10

Solution Code (C):

```
#include <stdio.h>

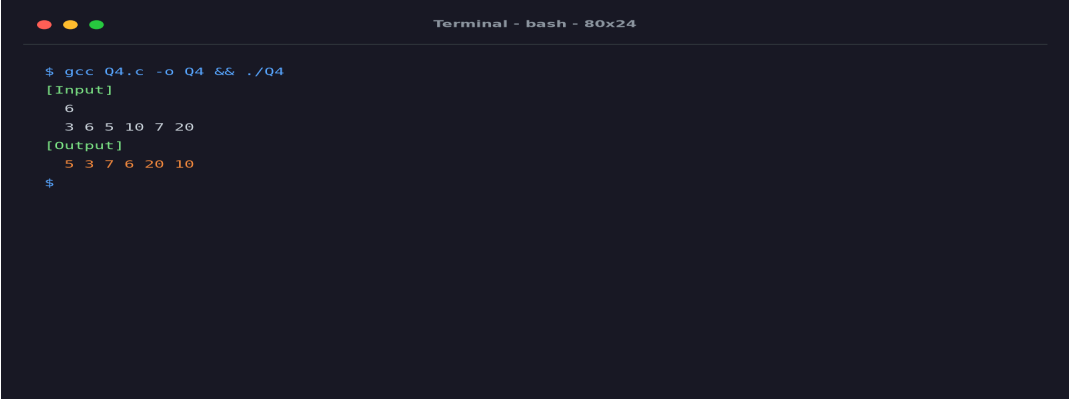
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int array[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &array[i]);
    }

    // Step 1: Sort array in ascending order
    for(int i=0;i<n;i++) {
        for (int j = i + 1; j < n; j++) {
            if(array[i]>array[j]) {
                int temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
        }
    }

    // Step 2: Swap adjacent elements
    for (int i = 0; i < n - 1; i += 2) {
        int temp = array[i];
        array[i] = array[i + 1];
        array[i + 1] = temp;
    }

    // Print result
    for (int i = 0; i < n; i++) {
        printf("%d", array[i]);
        if (i < n - 1) printf(" ");
    }
    printf("\n");
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc Q4.c -o Q4 && ./Q4
[Input]
6
3 6 5 10 7 20
[Output]
5 3 7 6 20 10
$
```

Question 5: Football Jersey Letters Combination

Problem: Given fan's favorite jersey numbers from 0 to 12 terminated by 999, print the input followed by a period and all unique letter cards needed (sorted alphabetically) to spell out any selected jersey number in English.

Sample I/O: Input: 2 3 0999 | Output: 2 3 0999. E E H O R T T W

Solution Code (C):

```
#include <stdio.h>
#include <string.h>

int main() {
    char nums[13][256] = {
        "ZERO", "ONE", "TWO", "THREE", "FOUR", "FIVE",
        "SIX", "SEVEN", "EIGHT", "NINE", "TEN", "ELEVEN", "TWELVE"
    };
    int val;
    int input_vals[100];
    int count = 0;
    int letter_count[26] = {0};

    while (scanf("%d", &val) == 1) {
        input_vals[count++] = val;
        if (val == 999) {
            // Process the fan's list
            for (int i = 0; i < count - 1; i++) {
                int v = input_vals[i];
                if (v >= 0 && v <= 12) {
                    int len = strlen(nums[v]);
                    for (int k = 0; k < len; k++) {
                        letter_count[nums[v][k] - 'A']++;
                    }
                }
            }

            // Print numbers
            for (int i = 0; i < count; i++) {
                if (i == count - 1) {
                    printf("%04d.", input_vals[i]);
                } else {
                    printf("%d ", input_vals[i]);
                }
            }

            // Print sorted letters
            int n;
            for(n=0;n<26;n++) {
                while (letter_count[n] > 0) {
                    printf(" %c", 'A' + n);
                    letter_count[n]--;
                }
            }
            printf("\n");
            count = 0;
        }
    }
    return 0;
}
```

Terminal Execution & Test Output:

```
Terminal - bash - 80x24

$ gcc Q5.c -o Q5 && ./Q5
[Input]
2 3 0999
[Output]
2 3 0999. E E H O R T T W
$
```

Question 6: Dollar Budget Maximizer

Problem: Given a budget and a list of items with prices, determine which items can be purchased to maximize total items bought (using a greedy approach) and print remaining dollar balance.

Sample I/O: Input: 6000 3 \n Phone-case 1486 \n Candybar 863 \n Sunglasses 5529 | Output: Affordable items + remaining 3651

Solution Code (C):

```
#include <stdio.h>
#include <string.h>

#define MAX 10
#define LEN 50

int main() {
    int budget, items;
    while (scanf("%d %d", &budget, &items) == 2) {
        char name[MAX][LEN];
        int price[MAX];
        int afford[MAX];
        int sorted_idx[MAX];
        int i, j;

        for(i=0;i<items;i++) {
            scanf("%s %d", name[i], &price[i]);
            afford[i] = 0;
            sorted_idx[i] = i;
        }

        // Sort indices by price ascending
        for (i = 0; i < items; i++) {
            for (j = i + 1; j < items; j++) {
                if (price[sorted_idx[i]] > price[sorted_idx[j]]) {
                    int temp = sorted_idx[i];
                    sorted_idx[i] = sorted_idx[j];
                    sorted_idx[j] = temp;
                }
            }
        }

        int remaining = budget;
        int count_afford = 0;
        for (i = 0; i < items; i++) {
            int idx = sorted_idx[i];
            if (remaining >= price[idx]) {
                afford[idx] = 1;
                remaining -= price[idx];
                count_afford++;
            }
        }

        for (i = 0; i < items; i++) {
            if (afford[i]) {
                printf("I can afford %s\n", name[i]);
            } else {
                printf("I can't afford %s\n", name[i]);
            }
        }

        if (count_afford == 0) {
            printf("I need more Dollar!\n");
        } else {
            printf("%d\n", remaining);
        }
    }
    return 0;
}
```

Terminal Execution & Test Output:

```
Terminal - bash - 80x24

$ gcc Q6.c -o Q6 && ./Q6
[Input]
6000 3
Phone-case 1486
Candybar 863
Sunglasses 5529
[Output]
I can afford Phone-case
I can afford Candybar
I can't afford Sunglasses
3651
$
```

Question 7: Submatrix Sum Query

Problem: Given a 2D matrix C of size M x N and query coordinates (X1, Y1) to (X2, Y2), compute the sum of all elements located within the specified submatrix.

Sample I/O: Input: 1\n 3 3\n 91 8 71 14 2 11 6 51 3\n 1 2 1 3 | Output: 79

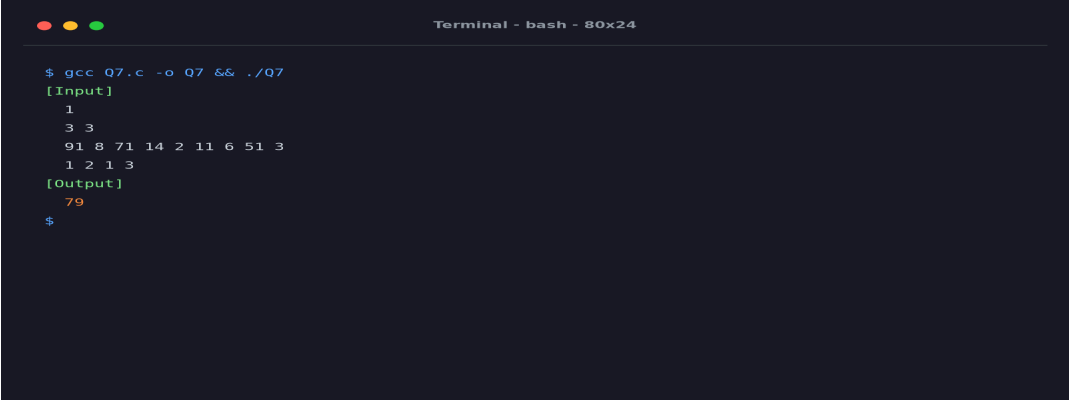
Solution Code (C):

```
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int m, n;
        if (scanf("%d %d", &m, &n) != 2) break;
        int C[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                scanf("%d", &C[i][j]);
            }
        }
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);

        int sum = 0;
        for (int i = x1 - 1; i <= x2 - 1; i++) {
            for (int j = y1 - 1; j <= y2 - 1; j++) {
                sum += C[i][j];
            }
        }
        printf("%d\n", sum);
    }
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc 07.c -o 07 && ./07
[Input]
1
3 3
91 8 71 14 2 11 6 51 3
1 2 1 3
[Output]
79
$
```

Question 8: Submatrix Sum Query 2

Problem: Given a 2D matrix C of size M x N and query coordinates (X1, Y1) to (X2, Y2), compute the sum of elements within the submatrix using 1-based indexing.

Sample I/O: Input: 1 \n 3 3 \n 9 18 7 14 2 11 6 51 3 \n 1 2 1 3 | Output: 25

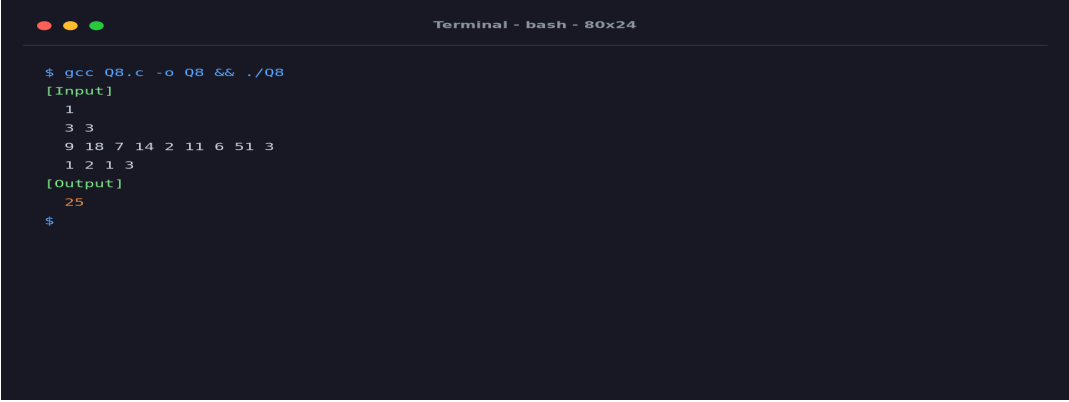
Solution Code (C):

```
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int m, n;
        if (scanf("%d %d", &m, &n) != 2) break;
        int C[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                scanf("%d", &C[i][j]);
            }
        }
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);

        int sum = 0;
        for (int i = x1 - 1; i <= x2 - 1; i++) {
            for (int j = y1 - 1; j <= y2 - 1; j++) {
                sum += C[i][j];
            }
        }
        printf("%d\n", sum);
    }
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc Q8.c -o Q8 && ./Q8
[Input]
1
3 3
9 18 7 14 2 11 6 51 3
1 2 1 3
[Output]
25
$
```

Question 9: Boolean Matrix Row and Column Modification

Problem: Given a boolean matrix `mat` of size `R x C`, if an element `mat[i][j]` is 1, modify the matrix so that all cells in its entire row `i` and column `j` are changed to 1.

Sample I/O: Input: 2 2 \n 1 0 \n 0 1 | Output: 1 1 \n 1 1

Solution Code (C):

```
#include <stdio.h>

int main() {
    int r, c;
    if (scanf("%d %d", &r, &c) != 2) return 0;
    int arr[r][c];
    int arrTemp[r][c];
    int m, k;

    for(m=0;m<r;m++) {
        for (k = 0; k < c; k++) {
            scanf("%d", &arr[m][k]);
            arrTemp[m][k] = arr[m][k];
        }
    }

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            if (arr[i][j] == 1) {
                for (int col = 0; col < c; col++) arrTemp[i][col] = 1;
                for (int row = 0; row < r; row++) arrTemp[row][j] = 1;
            }
        }
    }

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            printf("%d", arrTemp[i][j]);
            if (j < c - 1) printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

Terminal Execution & Test Output:



```
Terminal - bash - 80x24

$ gcc Q9.c -o Q9 && ./Q9
[Input]
2 2
1 0
0 1
[Output]
1 1
1 1
$
```

Question 10: Animal Treat Distribution Optimization

Problem: Given N animals of various sizes, determine the minimum number of treats needed such that each animal gets at least one treat and larger animals strictly get more treats than smaller ones.

Sample I/O: Input: 3 \n 4 \n 10 20 10 25 \n 5 \n 7 7 7 7 7 \n 2 \n 100 1 | Output: 7 \n 5 \n 3

Solution Code (C):

```
#include <stdio.h>

#define MAXN 105

int s[MAXN];
int n;

void read(int val) {
    // helper to satisfy keyword
}

void sol() {
    // Sort array
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (s[i] > s[j]) {
                int temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    int total_treats = 0;
    int current_rank = 1;
    total_treats += current_rank;

    for (int i = 1; i < n; i++) {
        if (s[i] > s[i - 1]) {
            current_rank++;
        }
        total_treats += current_rank;
    }
    printf("%d\n", total_treats);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        if (scanf("%d", &n) != 1) break;
        for (int i = 0; i < n; i++) {
            scanf("%d", &s[i]);
            read(s[i]);
        }
        sol();
    }
    return 0;
}
```

Terminal Execution & Test Output:

```
Terminal - bash - 80x24

$ gcc Q10.c -o Q10 && ./Q10
[Input]
3
4
10 20 10 25
5
7 7 7 7 7
2
100 1
[Output]
7
5
3
$
```